

Overfitting: Causes & How to Check

A diagnostic reference for regression and ML model engineering

The unifying signal

Overfitting shows up as a **gap between training and held-out performance**: high training performance with lower **validation** performance is the usual development signal. Almost every check below relies on holding out data the model never sees during training and watching that gap — often plotted as a curve over data size, model complexity, or training time. Reserve an untouched **test** set for a single final assessment of generalization rather than monitoring it repeatedly during development.

Cause	How to check	How to fix
Too many / irrelevant features	Compare train-vs-test gap with all features vs. a selected subset (hold everything else constant). If the gap shrinks and validation holds or improves, excess features were a contributor.	Domain-guided selection and regularization such as L1/elastic net; when fitting linear models, use multicollinearity diagnostics such as VIF. Drop redundant or irrelevant features.
Model too complex (too flexible)	Train models of increasing complexity (tree depth, polynomial degree) and plot train vs. validation error. If validation error rises while training error keeps falling, complexity is the culprit.	Simpler model, regularization (L1/L2), tree pruning, fewer layers/depth.
Too little training data	Plot a learning curve: performance vs. training-set size. If validation performance keeps improving materially as data is added, more representative data is likely to help.	Gather more representative data, data augmentation, or use a simpler model.
Training too long (iterative models)	Plot train and validation loss per epoch/iteration. The point where validation loss bottoms out and starts rising marks the onset of overfitting.	Early stopping at the validation-loss minimum.
Insufficient regularization	Sweep the regularization strength and watch the cross-validated curve. If a moderate increase in the penalty improves performance, the prior setting was likely under-regularized — keep tuning, since too much regularization underfits.	Tune L1/L2 penalty, dropout, or weight decay.
Data leakage (a source of misleading generalization estimates, not overfitting per se)	Suspiciously high validation or test performance is the tell. Audit whether any feature carries information unavailable at prediction time, or whether preprocessing was fit on the full dataset instead of the training fold.	Fit all preprocessing inside the CV loop; remove target-leaking features.
Hyperparameters tuned on test set	If settings were chosen by looking at test performance, the test score is optimistically biased. A substantially worse final test score is consistent with this, though sampling variability and distribution shift can also explain it.	Use a three-way train/validation/test split, or nested cross-validation.

Reading the gap

For **higher-is-better** metrics (accuracy, R^2 , AUC), the gap is **training score – validation score**. For **loss/error** metrics, where lower is better, the direction reverses — use **validation loss – training loss**. A large, consistent gap suggests overfitting; a small gap is desirable only when validation performance is also strong, since two poor-but-close scores indicate underfitting or a hard task. Estimate validation performance with an appropriate cross-validation scheme, and reserve an untouched test set for final evaluation.

Use the right split. For time series, grouped, or repeated observations, ordinary random cross-validation can leak information across folds — use time-aware or group-aware splits. Early stopping and any model selection should rely on validation folds, never on the final test set.

Note on isolating a cause: you can rarely prove a single cause — overfitting usually has several contributors that interact. Change one thing at a time (a controlled comparison) to measure each one's contribution. It is an experiment, not a proof.